

Go 生成式 AI 應用程式的實務觀測技巧

來源：<https://codelabs.developers.google.com/codelabs/gen-ai-o11y-for-devs/gen-ai-o11y-go?hl=zh-tw>

步驟數：13

目錄

1. [總覽](#)
2. [必要條件](#)
3. [專案設定](#)
4. [準備 Cloud Shell 編輯器](#)
5. [啟用 Google API](#)
6. [建立生成式 AI Go 應用程式](#)
7. [稽核 Vertex API 呼叫](#)
8. [記錄與生成式 AI 的互動](#)
9. [計算與生成式 AI 的互動次數](#)
10. [\(選用\) 使用 Open Telemetry 進行監控和追蹤](#)
11. [\(選用\) 記錄中經過模糊處理的機密資訊](#)
12. [\(選用\) 清除](#)
13. [恭喜](#)

1. 總覽

生成式 AI 應用程式與其他應用程式一樣，都需要可觀測性。[生成式 AI](#) 是否需要特殊的觀測技術？

在本實驗室中，您將建立簡單的生成式 AI 應用程式。將其部署至 [Cloud Run](#)。並使用 Google Cloud Observability 服務和產品，為其導入必要的監控和記錄功能。

學習目標

- 使用 [Cloud Shell 編輯器](#) 撰寫使用 Vertex AI 的應用程式
 - 在 GitHub 中儲存應用程式程式碼
 - 使用 [gcloud CLI](#) 將應用程式的原始碼部署至 Cloud Run
 - 在生成式 AI 應用程式中新增監控和記錄功能
 - 使用 [記錄指標](#)
 - 使用 Open Telemetry SDK 實作記錄和監控功能
 - 深入瞭解負責任的 AI 資料處理方式
-

步驟 2 · 約 0 分鐘

2. 必要條件

如果沒有 Google 帳戶，請[建立新帳戶](#)。

請改用個人帳戶，而非公司或學校帳戶。公司和學校帳戶可能設有限制，導致您無法啟用本實驗室所需的 API。

3. 專案設定

1. 使用 Google 帳戶登入 [Google Cloud 控制台](#)。
2. [建立新專案](#)，或選擇重複使用現有專案。記下您剛建立或選取的專案 ID。
3. 為專案[啟用計費功能](#)。

- 完成本實驗室的結算費用應低於 \$5 美元。
- 您可以按照本實驗室結尾的步驟刪除資源，以免產生後續費用。
- 新使用者可獲得價值 [\\$300 美元的免費試用期](#)。

4. 確認 Cloud Billing 的「我的專案」

已啟用計費功能

- 如果新專案的「Billing account」欄顯示 `Billing is disabled`，請按照下列步驟操作：
 1. 按一下「Actions」欄中的三點圖示
 2. 按一下「變更帳單」
 3. 選取要使用的帳單帳戶
- 如果您參加的是現場活動，帳戶名稱可能為「Google Cloud Platform 試用帳單帳戶」

如果你參加的是現場研討會，可獲得 [\\$5 美元抵免額](#)，做為這個實驗室的帳單帳戶。

注意：抵免額僅限研討會當天使用，且數量有限。

步驟 4 · 約 0 分鐘

4. 準備 Cloud Shell 編輯器

1. 前往 [Cloud Shell 編輯器](#)。如果系統顯示以下訊息，要求您授權 Cloud Shell 使用憑證呼叫 gcloud，請按一下「Authorize」（授權）繼續操作。

Authorize Cloud Shell

Authorize Cloud Shell to initialize the gcloud command with your credentials.

[Close](#) [Authorize](#)

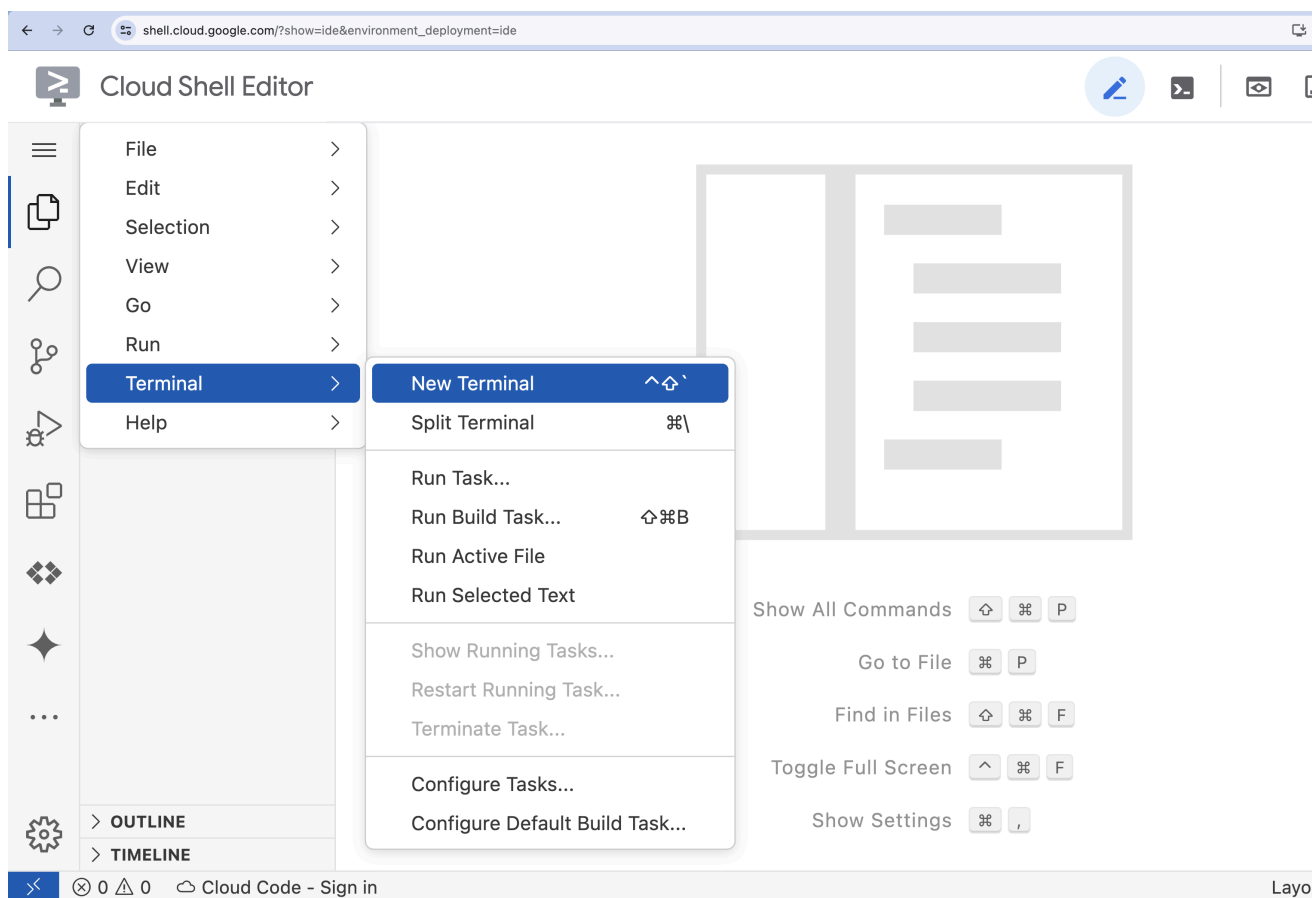
2. 開啟終端機視窗

1. 按一下漢堡選單



2. 按一下「終端機」。

3. 按一下「New Terminal」（新增終端機）



3. 在終端機中設定專案 ID：

```
gcloud config set project [PROJECT_ID]
```

將 `[PROJECT_ID]` 替換為專案 ID。舉例來說，如果專案 ID 為 `lab-example-project`，指令會是：

```
gcloud config set project lab-project-id-example
```

如果系統提示以下訊息，指出 gcloud 要求提供憑證給 GCPI API，請按一下「Authorize」(授權)繼續操作。

Authorize Cloud Shell

gcloud is requesting your credentials to make a GCP API call.

Click to authorize this and future calls that require your credentials.

REJECT

AUTHORIZE

執行成功後，您應該會看到以下訊息：

```
Updated property [core/project].
```

如果看到 `WARNING` 並收到 `Do you want to continue (Y/N)?` 提示，可能是輸入的專案 ID 有誤。按下 `N` 和 `Enter`，找到正確的專案 ID 後，再次嘗試執行 `gcloud config set project` 指令。

4. (選用) 如果您找不到專案 ID，請執行下列指令，查看所有專案的專案 ID，並依建立時間降序排序：

```
gcloud projects list \
  --format='value(projectId,createTime)' \
  --sort-by=~createTime
```

步驟 5 · 約 0 分鐘

5. 啟用 Google API

在終端機中，啟用本實驗室所需的 Google API：

```
gcloud services enable \  
  run.googleapis.com \  
  cloudbuild.googleapis.com \  
  aiplatform.googleapis.com \  
  logging.googleapis.com \  
  monitoring.googleapis.com \  
  cloudtrace.googleapis.com
```

這個指令需要一段時間才能完成。最後，系統會產生類似以下的成功訊息：

```
Operation "operations/acf.p2-73d90d00-47ee-447a-b600" finished successfully.
```

如果收到開頭為 `ERROR: (gcloud.services.enable) HttpError accessing` 的錯誤訊息，且包含下列錯誤詳細資料，請延遲 1 到 2 分鐘後重試指令。

```
"error": {  
  "code": 429,  
  "message": "Quota exceeded for quota metric 'Mutate requests' and limit 'Mutate requests per minute' of service 'serviceusage.googleapis.com' ...",  
  "status": "RESOURCE_EXHAUSTED",  
  ...  
}
```


6. 建立生成式 AI Go 應用程式

在這個步驟中，您會編寫簡單的應用程式程式碼，根據要求使用 [Gemini 模型](#) 顯示 10 個與所選動物相關的有趣事實。請按照下列步驟建立應用程式程式碼。

1. 在終端機中建立 `codelab-ol1y` 目錄：

```
mkdir ~/codelab-ol1y
```

2. 將目前目錄變更為 `codelab-ol1y`：

```
cd ~/codelab-ol1y
```

3. 初始化 Go 模組：

```
go mod init codelab
```

4. 安裝 Go 適用的 Vertex AI SDK：

```
go get cloud.google.com/go/vertexai/genai
```

5. 安裝 Go 的中繼資料程式庫，取得目前的專案 ID：

```
go get cloud.google.com/go/compute/metadata
```

6. 建立 `setup.go` 檔案，然後在 Cloud Shell 編輯器中開啟該檔案：

```
cloudshell edit setup.go
```

這個值區將用於託管初始化程式碼。編輯器視窗中會顯示名為 `setup.go` 的新空白檔案。

7. 複製下列程式碼，並貼到開啟的 `setup.go` 檔案中：

```
package main

import (
    "context"
    "os"

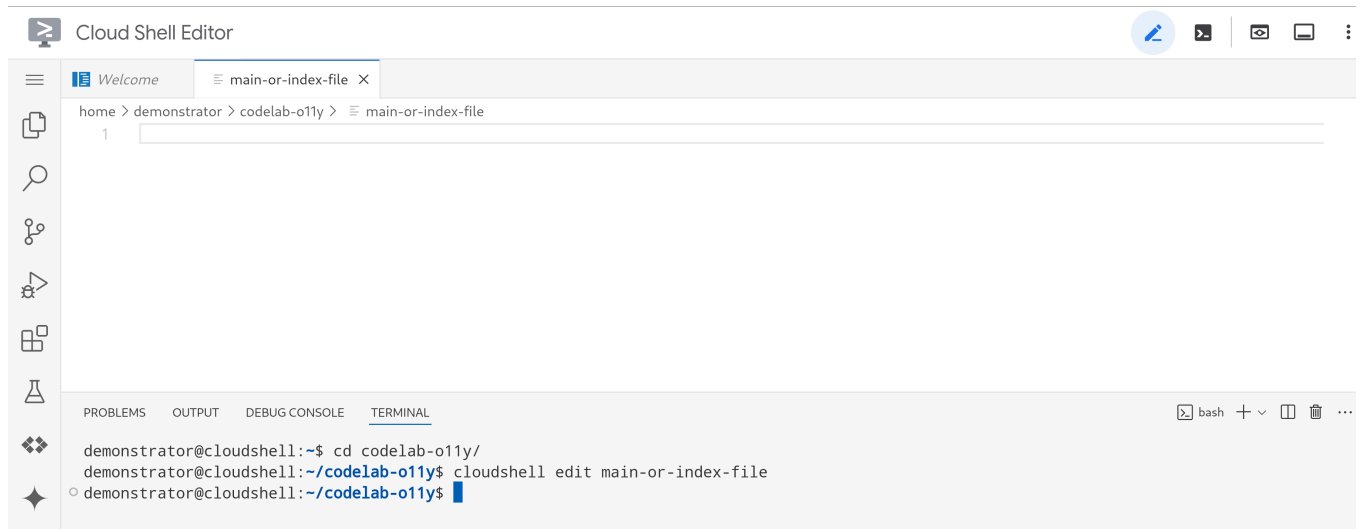
    "cloud.google.com/go/compute/metadata"
)

func projectID(ctx context.Context) (string, error) {
    var projectID = os.Getenv("GOOGLE_CLOUD_PROJECT")
    if projectID == "" {
        return metadata.ProjectIDWithContext(ctx)
    }
    return projectID, nil
}
```

8. 返回終端機視窗，然後執行下列指令，在 Cloud Shell 編輯器中建立並開啟 `main.go` 檔案：

```
cloudshell edit main.go
```

現在編輯器視窗中應該會顯示空白檔案 (位於終端機上方)。畫面應如下所示：



9. 複製下列程式碼，並貼到開啟的 `main.go` 檔案中：

```

package main

import (
    "context"
    "fmt"
    "net/http"
    "os"

    "cloud.google.com/go/vertexai/genai"
)

var model *genai.GenerativeModel

func main() {
    ctx := context.Background()
    projectID, err := projectID(ctx)
    if err != nil {
        return
    }

    var client *genai.Client
    client, err = genai.NewClient(ctx, projectID, "us-central1")
    if err != nil {
        return
    }
    defer client.Close()
    model = client.GenerativeModel("gemini-1.5-flash-001")
    http.HandleFunc("/", Handler)
    port := os.Getenv("PORT")
    if port == "" {
        port = "8080"
    }
    if err := http.ListenAndServe(":"+port, nil); err != nil {
        return
    }
}

func Handler(w http.ResponseWriter, r *http.Request) {
    animal := r.URL.Query().Get("animal")
    if animal == "" {
        animal = "dog"
    }

    prompt := fmt.Sprintf("Give me 10 fun facts about %s. Return the results as HTML without markdown backticks.", animal)
    resp, err := model.GenerateContent(r.Context(), genai.Text(prompt))
    if err != nil {
        w.WriteHeader(http.StatusTooManyRequests)
        return
    }

    if len(resp.Candidates) > 0 && len(resp.Candidates[0].Content.Parts) > 0 {

```

```

        htmlContent := resp.Candidates[0].Content.Parts[0]
        w.Header().Set("Content-Type", "text/html; charset=utf-8")
        fmt.Fprint(w, htmlContent)
    }
}

```

幾秒後，Cloud Shell 編輯器會自動儲存程式碼。

將生成式 AI 應用程式的程式碼部署至 Cloud Run

1. 在終端機視窗中執行指令，將應用程式的原始碼部署至 Cloud Run。

```

gcloud run deploy codelab-olly-service \
  --source="${HOME}/codelab-olly/" \
  --region=us-central1 \
  --allow-unauthenticated

```

如果看到如下提示，表示指令會建立新的存放區。按一下 `Enter`。

```

Deploying from source requires an Artifact Registry Docker repository to store
built containers.
A repository named [cloud-run-source-deploy] in region [us-central1] will be
created.

Do you want to continue (Y/n)?

```

部署程序最多可能需要幾分鐘才能完成。部署程序完成後，您會看到類似以下的輸出內容：

```

Service [codelab-olly-service] revision [codelab-olly-service-00001-t2q] has been
deployed and is serving 100 percent of traffic.
Service URL: https://codelab-olly-service-12345678901.us-central1.run.app

```

2. 將顯示的 Cloud Run 服務網址複製到瀏覽器的另一個分頁或視窗。或者，您也可以在終端機中執行下列指令，列印服務網址，然後按住 **Ctrl** 鍵並點選顯示的網址，開啟該網址：

```

gcloud run services list \
  --format='value(URL)' \
  --filter='SERVICE:"codelab-olly-service"'

```

開啟網址時，您可能會收到 500 錯誤訊息，或看到以下訊息：

```

Sorry, this is just a placeholder...

```

這表示服務未完成部署作業。請稍候片刻，然後重新整理頁面。最後你會看到以「Fun Dog Facts」(狗狗趣味小知識)開頭的文字，其中包含 10 個狗狗趣味小知識。

嘗試與應用程式互動，瞭解各種動物的有趣知識。如要這麼做，請將 `animal` 參數附加至網址，例如 `?animal=[ANIMAL]`，其中 `[ANIMAL]` 是動物名稱。舉例來說，附加 `?animal=cat` 即可取得 10 個關於貓咪的趣味知識，附加 `?`

`animal=sea turtle` 則可取得 10 個關於海龜的趣味知識。

請保留這個服務網址分頁 (或視窗)，因為後續步驟會用到。

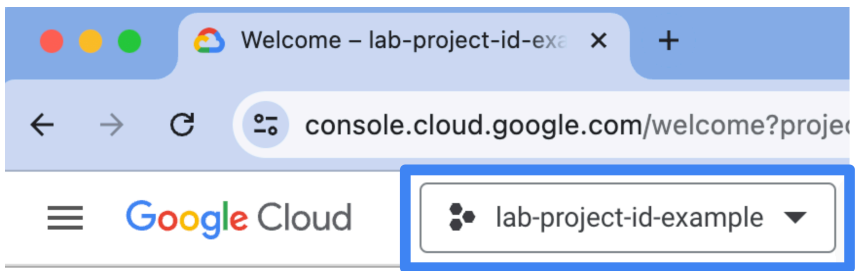
7. 稽核 Vertex API 呼叫

稽核 Google API 呼叫可回答「誰在何時何地呼叫特定 API？」等問題。在排解應用程式問題、調查資源耗用情形或執行軟體鑑識分析時，稽核作業非常重要。

[稽核記錄](#)可讓您追蹤管理員和系統活動，以及記錄對「資料讀取」和「資料寫入」API 作業的呼叫。如要稽核生成內容的 Vertex AI 要求，您必須在 Cloud 控制台中[啟用「資料讀取」稽核記錄](#)。

啟用「資料讀取」稽核記錄可能會產生額外費用。

- 按一下下方按鈕，在 Cloud 控制台中開啟「稽核記錄」頁面
- 確認頁面已選取您為這個實驗室建立的專案。所選專案會顯示在頁面左上角，漢堡選單的右側：



如有需要，請從下拉式方塊選取正確的專案。

- 在「資料存取稽核記錄設定」表格的「服務」欄中，找出 `Vertex AI API` 服務，然後選取服務名稱左側的核取方塊，選取該服務。

Data Access audit logs configuration

The effective data access configuration below combines the configuration for the currently selected resource and the data access configurations set on all parent resources.

Filter Vertex AI API Enter property name or value						
☒	Service ↑	Admin Read	Data Read	Data Write	Exempted principals	Inherited exempted principals
☒	Vertex AI API				0	0

- 在右側的資訊面板中，選取「資料讀取」稽核類型。

PERMISSION TYPES

EXEMPTED PRINCIPALS

You can configure what types of operations are recorded in your Data Access audit logs for the selected services. There are several subtypes of Data Access audit logs:

☐

Admin Read

Records operations that read metadata or configuration information.

☒

Data Read

Records operations that read user-provided data.

☐

Data Write

Records operations that write user-provided data.

SAVE

5. 按一下 [儲存]。

如要產生稽核記錄，請開啟服務網址。重新整理頁面，同時變更 `?animal=` 參數的值，即可取得不同的結果。

瞭解稽核記錄

- 按一下下方按鈕，在 Cloud 控制台中開啟「記錄檔探索工具」頁面：
- 將下列篩選器貼到「查詢」窗格。

```
LOG_ID("cloudaudit.googleapis.com%2Fdata_access") AND  
protoPayload.serviceName="aiplatform.googleapis.com"
```

「查詢」窗格是位於「Logs Explorer」頁面頂端的編輯器：

Logs Explorer

Query libraryShare linkPreferences

Last 1 hourPDT

Learn

Project logs

Search all fields

Run query

All resources

All log names

All severities

Correlate by

+2 filters

Show query

1 LOG_ID("cloudaudit.googleapis.com%2Fdata_access") AND

2 protoPayload.serviceName="aiplatform.googleapis.com"

Log fields

Timeline

400

- 點選「執行查詢」
- 選取其中一個稽核記錄項目，然後展開欄位，檢查記錄中擷取的資訊。
您可以查看 Vertex API 呼叫的詳細資料，包括使用的方法和模型。您也可以查看呼叫者的身分，以及授權呼叫的權限。

8. 記錄與生成式 AI 的互動

您不會在稽核記錄中找到 API 要求參數或回應資料。不過，這項資訊對於排解應用程式和工作流程分析問題可能非常重要。在本步驟中，我們將新增應用程式記錄，填補這項缺口。記錄功能會使用標準 Go `log/slog` 套件寫入結構化記錄。`log/slog` 套件不知道如何將記錄寫入 Google Cloud。這項功能支援寫入標準輸出內容。不過，Cloud Run [功能](#)會擷取列印至標準輸出的資訊，並自動將其擷取至 Cloud Logging。如要正確擷取結構化記錄，列印的記錄必須符合格式。請按照下列操作說明，在 Go 應用程式中新增結構化記錄功能。

1. 返回瀏覽器中的「Cloud Shell」視窗 (或分頁)。
2. 在終端機中重新開啟 `setup.go`：

```
cloudshell edit ~/codelab-olly/setup.go
```

3. 將程式碼換成設定記錄的版本。如要取代程式碼，請刪除檔案內容，然後複製下列程式碼並貼到編輯器中：

```

package main

import (
    "context"
    "os"
    "log/slog"
    "cloud.google.com/go/compute/metadata"
)

func projectID(ctx context.Context) (string, error) {
    var projectID = os.Getenv("GOOGLE_CLOUD_PROJECT")
    if projectID == "" {
        return metadata.ProjectIDWithContext(ctx)
    }
    return projectID, nil
}

func setupLogging() {
    opts := &slog.HandlerOptions{
        Level: slog.LevelDebug,
        ReplaceAttr: func(group []string, a slog.Attr) slog.Attr {
            switch a.Key {
            case slog.LevelKey:
                a.Key = "severity"
                if level := a.Value.Any().(slog.Level); level == slog.LevelWarn {
                    a.Value = slog.StringValue("WARNING")
                }
            case slog.MessageKey:
                a.Key = "message"
            case slog.TimeKey:
                a.Key = "timestamp"
            }
            return a
        },
    }
    jsonHandler := slog.NewJSONHandler(os.Stdout, opts)
    slog.SetDefault(slog.New(jsonHandler))
}

```

4. 返回終端機，然後重新開啟 `main.go`：

```
cloudshell edit ~/codelab-011y/main.go
```

5. 將應用程式程式碼替換為可記錄與模型互動的程式碼版本。如要取代程式碼，請刪除檔案內容，然後複製下列程式碼並貼到編輯器中：

```

package main

import (
    "context"
    "fmt"
    "net/http"
    "os"

    "encoding/json"
    "log/slog"

    "cloud.google.com/go/vertexai/genai"
)

var model *genai.GenerativeModel

func main() {
    ctx := context.Background()
    projectID, err := projectID(ctx)
    if err != nil {
        return
    }

    setupLogging()

    var client *genai.Client
    client, err = genai.NewClient(ctx, projectID, "us-central1")
    if err != nil {
        slog.ErrorContext(ctx, "Failed to marshal response to JSON", slog.Any("error",
err))
        os.Exit(1)
    }
    defer client.Close()
    model = client.GenerativeModel("gemini-1.5-flash-001")
    http.HandleFunc("/", Handler)
    port := os.Getenv("PORT")
    if port == "" {
        port = "8080"
    }
    if err := http.ListenAndServe(":"+port, nil); err != nil {
        slog.ErrorContext(ctx, "Failed to start the server", slog.Any("error", err))
        os.Exit(1)
    }
}

func Handler(w http.ResponseWriter, r *http.Request) {
    animal := r.URL.Query().Get("animal")
    if animal == "" {
        animal = "dog"
    }

    prompt := fmt.Sprintf("Give me 10 fun facts about %s. Return the results as HTML

```

```

without markdown backticks.", animal)
    resp, err := model.GenerateContent(r.Context(), genai.Text(prompt))
    if err != nil {
        w.WriteHeader(http.StatusTooManyRequests)
        return
    }

    jsonBytes, err := json.Marshal(resp)
    if err != nil {
        slog.Error("Failed to marshal response to JSON", slog.Any("error", err))
    } else {
        slog.DebugContext(r.Context(), "content is generated", slog.String("animal",
animal),
            slog.String("prompt", prompt), slog.String("response", string(jsonBytes)))
    }

    if len(resp.Candidates) > 0 && len(resp.Candidates[0].Content.Parts) > 0 {
        htmlContent := resp.Candidates[0].Content.Parts[0]
        w.Header().Set("Content-Type", "text/html; charset=utf-8")
        fmt.Fprint(w, htmlContent)
    }
}

```

記錄檔會設定為列印至 `stdout`，並由 Cloud Run 記錄代理程式收集，然後非同步擷取至 Cloud Logging。 `main()` 函式經過修改後，會設定 Go 標準結構化記錄，以使用符合[結構化格式設定指南](#)的 JSON 結構定義。所有 `return` 陳述式都會替換為在結束前寫入錯誤記錄的程式碼。 `Handler()` 函式會經過檢測，在收到 Vertex AI API 呼叫的回應時，寫入結構化記錄。記錄會擷取要求中的動物參數，以及模型的提示和回覆。

幾秒後，Cloud Shell 編輯器會自動儲存變更。

將生成式 AI 應用程式的程式碼部署至 Cloud Run

1. 在終端機視窗中執行指令，將應用程式的原始碼部署至 Cloud Run。

```

gcloud run deploy codelab-olly-service \
    --source="${HOME}/codelab-olly/" \
    --region=us-central1 \
    --allow-unauthenticated

```

如果看到如下提示，表示指令會建立新的存放區。按一下 `Enter`。

```

Deploying from source requires an Artifact Registry Docker repository to store
built containers.
A repository named [cloud-run-source-deploy] in region [us-central1] will be
created.

Do you want to continue (Y/n)?

```

部署程序最多可能需要幾分鐘才能完成。部署程序完成後，您會看到類似以下的輸出內容：

```
Service [codelab-olly-service] revision [codelab-olly-service-00001-t2q] has been
deployed and is serving 100 percent of traffic.
Service URL: https://codelab-olly-service-12345678901.us-central1.run.app
```

2. 將顯示的 Cloud Run 服務網址複製到瀏覽器的另一個分頁或視窗。或者，您也可以在終端機中執行下列指令，列印服務網址，然後按住 **Ctrl** 鍵並點選顯示的網址，開啟該網址：

```
gcloud run services list \
  --format='value(URL)' \
  --filter='SERVICE:"codelab-olly-service"'
```

開啟網址時，您可能會收到 500 錯誤訊息，或看到以下訊息：

```
Sorry, this is just a placeholder...
```

這表示服務未完成部署作業。請稍候片刻，然後重新整理頁面。最後你會看到以「Fun Dog Facts」(狗狗趣味小知識)開頭的文字，其中包含 10 個狗狗趣味小知識。

如要產生應用程式記錄，請開啟服務網址。重新整理頁面，同時變更 `?animal=` 參數的值，即可取得不同的結果。如要查看應用程式記錄，請執行下列操作：

1. 按一下下方按鈕，在 Cloud 控制台中開啟記錄檔探索工具頁面：
2. 將下列篩選條件貼到「查詢」窗格 ([記錄檔探索工具介面](#)中的 #2)：

```
LOG_ID("run.googleapis.com%2Fstdout") AND
severity=DEBUG
```

3. 點選「執行查詢」

查詢結果會顯示記錄，包括提示和 Vertex AI 回覆，以及[安全評分](#)。

請注意，記錄的 Vertex API 提示和回覆可能含有不適合觀察的私密資訊。

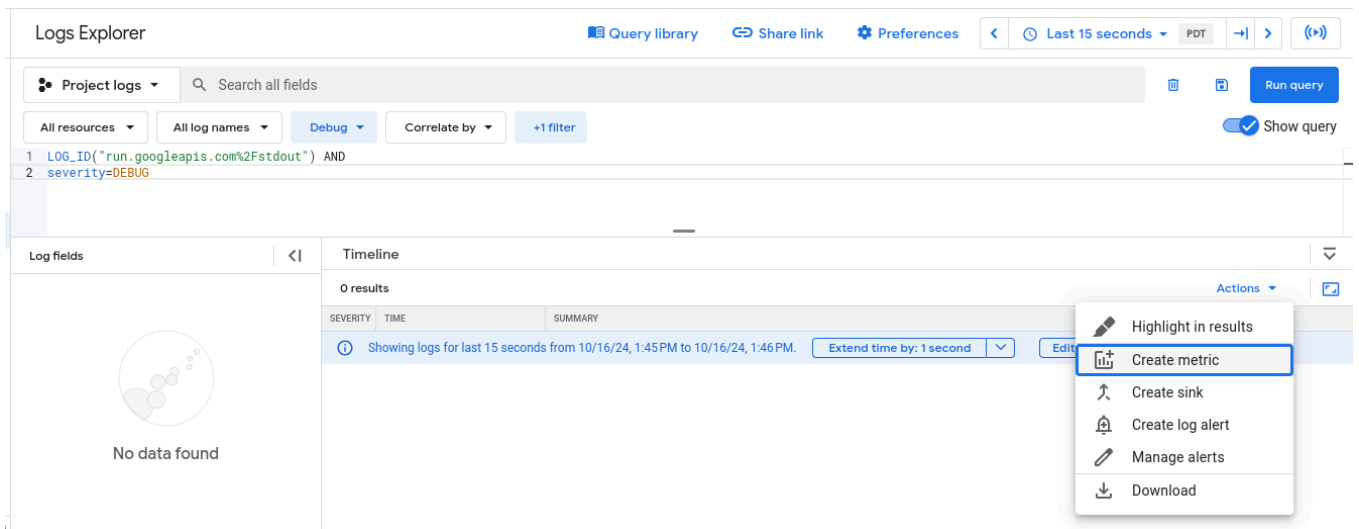
步驟 9 · 約 0 分鐘

9. 計算與生成式 AI 的互動次數

Cloud Run 會寫入**代管指標**，可用於監控已部署的服務。使用者管理的監控指標可進一步控管資料，以及指標更新的頻率。如要實作這類指標，必須編寫程式碼來收集資料，並將資料寫入 **Cloud Monitoring**。如要瞭解如何使用 OpenTelemetry SDK 實作，請參閱下一個 (選用) 步驟。

這個步驟說明實作使用者指標的替代做法，也就是**記錄指標**。記錄指標可讓您從應用程式寫入 Cloud Logging 的記錄項目產生監控指標。我們將使用上一個步驟中實作的應用程式記錄，定義**計數器類型**的記錄型指標。這項指標會計算 Vertex API 的成功呼叫次數。

1. 查看上一步使用的「記錄檔探索器」視窗。在「查詢」窗格下方找到「動作」下拉式選單，然後點選開啟。請參閱下方螢幕截圖，找出選單：



2. 在開啟的選單中選取「建立指標」，開啟「建立記錄指標」面板。
3. 請按照下列步驟，在「建立記錄指標」面板中設定新的計數器指標：
 1. 設定「指標類型」：選取「計數器」。
 2. 在「詳細資料」部分中，設定下列欄位：
 - **記錄指標名稱**：將名稱設為 `model_interaction_count`。存在一些命名限制；詳情請參閱**疑難排解**一文。
 - **說明**：輸入指標說明。例如：`Number of log entries capturing successful call to model inference.`
 - **單位**：保留此欄位空白，或插入數字 `1`。
 3. 保留「篩選器選取」部分的值。請注意，「Build filter」(建立篩選器) 欄位與我們用來查看應用程式記錄的篩選器相同。
 4. (選用) 新增標籤，方便計算每種動物的叫聲次數。注意：這個標籤可能會大幅增加指標的**基數**，因此不建議用於正式環境：
 1. 按一下 [Add label] (新增標籤)。
 2. 在「標籤」部分中設定下列欄位：
 - **標籤名稱**：將名稱設為 `animal`。
 - **說明**：輸入標籤說明。例如：`Animal parameter`。
 - **標籤類型**：選取 `STRING`。
 - **欄位名稱**：輸入 `jsonPayload.animal`。

- 規則運算式：留空。

3. 然後按一下 [完成]。

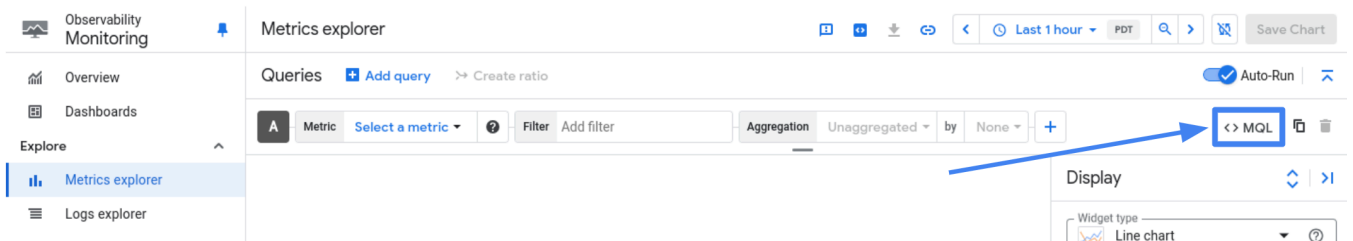
5. 按一下「建立指標」即可建立指標。

您也可以使用 `gcloud logging metrics create` [CLI 指令](#) 或 `google_logging_metric` [Terraform 資源](#)，從「記錄指標」頁面建立記錄指標。

如要產生指標資料，請開啟服務網址。多次重新整理開啟的頁面，對模型發出多個呼叫。和先前一樣，請嘗試在參數中使用不同動物。

輸入 PromQL 查詢，搜尋記錄指標資料。如要輸入 PromQL 查詢，請按照下列步驟操作：

1. 按一下下方按鈕，在 Cloud 控制台中開啟 Metrics Explorer 頁面：
2. 在查詢建構工具窗格的工具列中，選取名稱為「<> MQL」或「<> PromQL」的按鈕。如要瞭解按鈕位置，請參閱下圖。

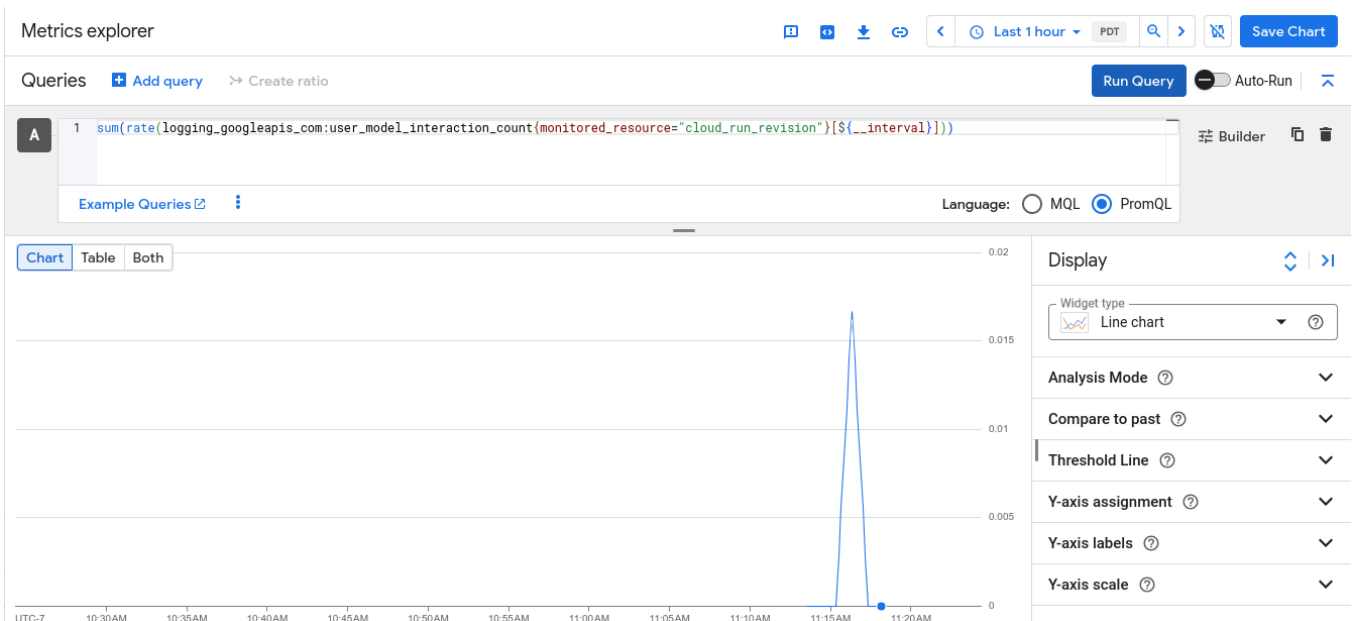


3. 確認「語言」切換按鈕已選取「PromQL」。語言切換鍵位於可格式化查詢的工具列中。
4. 在「查詢」編輯器中輸入查詢：

```
sum(rate(logging_googleapis_com:user_model_interaction_count{monitored_resource="cloud_run_revision"}[${__interval}])))
```

如要進一步瞭解如何使用 PromQL，請參閱「[在 Cloud Monitoring 中使用 PromQL](#)」。

5. 按一下 [Run query] (執行查詢)，您會看到類似以下螢幕截圖的折線圖：



請注意，啟用「自動執行」切換按鈕後，系統就不會顯示「執行查詢」按鈕。

10. (選用) 使用 Open Telemetry 進行監控和追蹤

如上一步所述，您可以使用 OpenTelemetry (Otel) SDK 導入指標。建議在微服務架構中使用 OTel。這個步驟說明下列事項：

- 初始化 OTel 元件，支援追蹤及監控應用程式
- 使用 Cloud Run 環境的資源中繼資料填入 OTel 設定
- 使用自動追蹤功能檢測 Flask 應用程式
- 導入計數器指標，監控模型呼叫成功次數
- 將追蹤記錄與應用程式記錄檔建立關聯

建議您使用 [OTel 收集器](#)，收集及擷取一或多項服務的所有可觀測性資料，做為產品層級服務的架構。為簡化程序，這個步驟中的程式碼不會使用收集器。而是使用 OTel 匯出功能，將資料直接寫入 Google Cloud。

設定 OTel 元件，以追蹤及監控指標

1. 返回瀏覽器中的「Cloud Shell」視窗 (或分頁)。
2. 在終端機中重新開啟 `setup.go`：

```
cloudshell edit ~/codelab-011y/setup.go
```

3. 將程式碼替換為初始化 OpenTelemetry 追蹤記錄和指標收集作業的版本。如要取代程式碼，請刪除檔案內容，然後複製下列程式碼並貼到編輯器中：

```

package main

import (
    "context"
    "errors"
    "fmt"
    "net/http"
    "os"

    "log/slog"

    "go.opentelemetry.io/contrib/detectors/gcp"
    "go.opentelemetry.io/contrib/instrumentation/net/http/otelhttp"
    "go.opentelemetry.io/contrib/propagators/autoprop"
    "go.opentelemetry.io/otel"
    sdkmetric "go.opentelemetry.io/otel/sdk/metric"
    "go.opentelemetry.io/otel/sdk/resource"
    sdktrace "go.opentelemetry.io/otel/sdk/trace"
    semconv "go.opentelemetry.io/otel/semconv/v1.27.0"
    "go.opentelemetry.io/otel/trace"

    cloudmetric "github.com/GoogleCloudPlatform/opentelemetry-operations-
go/exporter/metric"
    cloudtrace "github.com/GoogleCloudPlatform/opentelemetry-operations-
go/exporter/trace"

    "cloud.google.com/go/compute/metadata"
)

var (
    projID string
)

func projectID(ctx context.Context) (string, error) {
    var projectID = os.Getenv("GOOGLE_CLOUD_PROJECT")
    if projectID == "" {
        return metadata.ProjectIDWithContext(ctx)
    }
    return projectID, nil
}

func setupLogging() {
    opts := &slog.HandlerOptions{
        Level: slog.LevelDebug,
        ReplaceAttr: func(group []string, a slog.Attr) slog.Attr {
            switch a.Key {
            case slog.LevelKey:
                a.Key = "severity"
                if level := a.Value.Any().(slog.Level); level == slog.LevelWarn {
                    a.Value = slog.StringValue("WARNING")
                }
            case slog.MessageKey:

```

```

        a.Key = "message"
    case slog.TimeKey:
        a.Key = "timestamp"
    }
    return a
},
}

jsonHandler := slog.NewJSONHandler(os.Stdout, opts)
instrumentedHandler := handlerWithSpanContext(jsonHandler)
slog.SetDefault(slog.New(instrumentedHandler))
}

type spanContextLogHandler struct {
    slog.Handler
}

func handlerWithSpanContext(handler slog.Handler) *spanContextLogHandler {
    return &spanContextLogHandler{Handler: handler}
}

func (t *spanContextLogHandler) Handle(ctx context.Context, record slog.Record) error {
    if s := trace.SpanContextFromContext(ctx); s.IsValid() {
        trace := fmt.Sprintf("projects/%s/traces/%s", projID, s.TraceID())
        record.AddAttrs(
            slog.Any("logging.googleapis.com/trace", trace),
        )
        record.AddAttrs(
            slog.Any("logging.googleapis.com/spanId", s.SpanID()),
        )
        record.AddAttrs(
            slog.Bool("logging.googleapis.com/trace_sampled",
s.TraceFlags().IsSampled()),
        )
    }
    return t.Handler.Handle(ctx, record)
}

func setupTelemetry(ctx context.Context) (shutdown func(context.Context) error, err
error) {
    var shutdownFuncs []func(context.Context) error
    shutdown = func(ctx context.Context) error {
        var err error
        for _, fn := range shutdownFuncs {
            err = errors.Join(err, fn(ctx))
        }
        shutdownFuncs = nil
        return err
    }

    projID, err = projectID(ctx)
    if err != nil {
        err = errors.Join(err, shutdown(ctx))
    }
}

```

```

        return
    }

    res, err2 := resource.New(
        ctx,
        resource.WithDetectors(gcp.NewDetector()),
        resource.WithTelemetrySDK(),
        resource.WithAttributes(semconv.ServiceNameKey.String(os.Getenv("K_SERVICE"))),
    )
    if err2 != nil {
        err = errors.Join(err2, shutdown(ctx))
        return
    }

    otel.SetTextMapPropagator(autoprop.NewTextMapPropagator())

    texporter, err2 := cloudtrace.New(cloudtrace.WithProjectID(projID))
    if err2 != nil {
        err = errors.Join(err2, shutdown(ctx))
        return
    }
    tp := sdktrace.NewTracerProvider(
        sdktrace.WithSampler(sdktrace.AlwaysSample()),
        sdktrace.WithResource(res),
        sdktrace.WithBatcher(texporter))
    shutdownFuncs = append(shutdownFuncs, tp.Shutdown)
    otel.SetTracerProvider(tp)

    mexporter, err2 := cloudmetric.New(cloudmetric.WithProjectID(projID))
    if err2 != nil {
        err = errors.Join(err2, shutdown(ctx))
        return
    }
    mp := sdkmetric.NewMeterProvider(
        sdkmetric.WithReader(sdkmetric.NewPeriodicReader(mexporter)),
        sdkmetric.WithResource(res),
    )
    shutdownFuncs = append(shutdownFuncs, mp.Shutdown)
    otel.SetMeterProvider(mp)

    return shutdown, nil
}

func registerHttpHandler(route string, handleFn http.HandlerFunc) {
    instrumentedHandler := otelhttp.NewHandler(otelhttp.WithRouteTag(route, handleFn),
route)
    http.Handle(route, instrumentedHandler)
}

```

4. 返回終端機，然後執行下列指令，更新 `go.mod` 檔案中的 Go 模組定義：

```
go mod tidy
```

5. 返回終端機，然後重新開啟 `main.go`：

```
cloudshell edit ~/codelab-ol1y/main.go
```

6. 將目前的程式碼換成可監控 HTTP 追蹤並寫入效能指標的版本。如要取代程式碼，請刪除檔案內容，然後複製下列程式碼並貼到編輯器中：

```

package main

import (
    "context"
    "errors"
    "fmt"
    "net/http"
    "os"

    "encoding/json"
    "log/slog"

    "cloud.google.com/go/vertexai/genai"

    "go.opentelemetry.io/otel"
    "go.opentelemetry.io/otel/attribute"
    "go.opentelemetry.io/otel/metric"
)

var model *genai.GenerativeModel
var counter metric.Int64Counter

const scopeName = "genai-olly/go/workshop/example"

func main() {
    ctx := context.Background()
    projectID, err := projectID(ctx)
    if err != nil {
        return
    }

    setupLogging()
    shutdown, err := setupTelemetry(ctx)
    if err != nil {
        slog.ErrorContext(ctx, "error setting up OpenTelemetry", slog.Any("error", err))
        os.Exit(1)
    }
    meter := otel.Meter(scopeName)
    counter, err = meter.Int64Counter("model_call_counter")
    if err != nil {
        slog.ErrorContext(ctx, "error setting up OpenTelemetry", slog.Any("error", err))
        os.Exit(1)
    }

    var client *genai.Client
    client, err = genai.NewClient(ctx, projectID, "us-central1")
    if err != nil {
        slog.ErrorContext(ctx, "Failed to marshal response to JSON", slog.Any("error",
err))
        os.Exit(1)
    }
    defer client.Close()

```

```

model = client.GenerativeModel("gemini-1.5-flash-001")

registerHttpHandler("/", Handler)

port := os.Getenv("PORT")
if port == "" {
    port = "8080"
}

if err = errors.Join(http.ListenAndServe(":"+port, nil), shutdown(ctx)); err != nil {
    slog.ErrorContext(ctx, "Failed to start the server", slog.Any("error", err))
    os.Exit(1)
}
}

func Handler(w http.ResponseWriter, r *http.Request) {
    animal := r.URL.Query().Get("animal")
    if animal == "" {
        animal = "dog"
    }

    prompt := fmt.Sprintf("Give me 10 fun facts about %s. Return the results as HTML without markdown backticks.", animal)
    resp, err := model.GenerateContent(r.Context(), genai.Text(prompt))
    if err != nil {
        w.WriteHeader(http.StatusTooManyRequests)
        return
    }
    jsonBytes, err := json.Marshal(resp)
    if err != nil {
        slog.ErrorContext(r.Context(), "Failed to marshal response to JSON", slog.Any("error", err))
    } else {
        slog.DebugContext(r.Context(), "content is generated", slog.String("animal", animal),
            slog.String("prompt", prompt), slog.String("response", string(jsonBytes)))
    }
    if len(resp.Candidates) > 0 && len(resp.Candidates[0].Content.Parts) > 0 {
        clabels := []attribute.KeyValue{attribute.Key("animal").String(animal)}
        counter.Add(r.Context(), 1, metric.WithAttributes(clabels...))
        htmlContent := resp.Candidates[0].Content.Parts[0]
        w.Header().Set("Content-Type", "text/html; charset=utf-8")
        fmt.Fprint(w, htmlContent)
    }
}
}

```

應用程式現在會使用 OpenTelemetry SDK，透過追蹤功能監控程式碼執行作業，並將成功執行的次數做為指標。 `main()` 方法已修改，可設定追蹤記錄和指標的 OpenTelemetry 匯出工具，直接寫入 Google Cloud Tracing 和 Monitoring。此外，這項服務還會執行額外設定，以 Cloud Run 環境相關中繼資料填入收集到的追蹤記錄和指標。 `Handler()` 函式已更新，每次 Vertex AI API 呼叫傳回有效結果時，指標計數器就會遞增。

幾秒後，Cloud Shell 編輯器會自動儲存變更。

將生成式 AI 應用程式的程式碼部署至 Cloud Run

1. 在終端機視窗中執行指令，將應用程式的原始碼部署至 Cloud Run。

```
gcloud run deploy codelab-olly-service \
  --source="${HOME}/codelab-olly/" \
  --region=us-central1 \
  --allow-unauthenticated
```

如果看到如下提示，表示指令會建立新的存放區。按一下 `Enter`。

```
Deploying from source requires an Artifact Registry Docker repository to store
built containers.
A repository named [cloud-run-source-deploy] in region [us-central1] will be
created.

Do you want to continue (Y/n)?
```

部署程序最多可能需要幾分鐘才能完成。部署程序完成後，您會看到類似以下的輸出內容：

```
Service [codelab-olly-service] revision [codelab-olly-service-00001-t2q] has been
deployed and is serving 100 percent of traffic.
Service URL: https://codelab-olly-service-12345678901.us-central1.run.app
```

2. 將顯示的 Cloud Run 服務網址複製到瀏覽器的另一個分頁或視窗。或者，您也可以在終端機中執行下列指令，列印服務網址，然後按住 **Ctrl** 鍵並點選顯示的網址，開啟該網址：

```
gcloud run services list \
  --format='value(URL)' \
  --filter='SERVICE:"codelab-olly-service"'
```

開啟網址時，您可能會收到 500 錯誤訊息，或看到以下訊息：

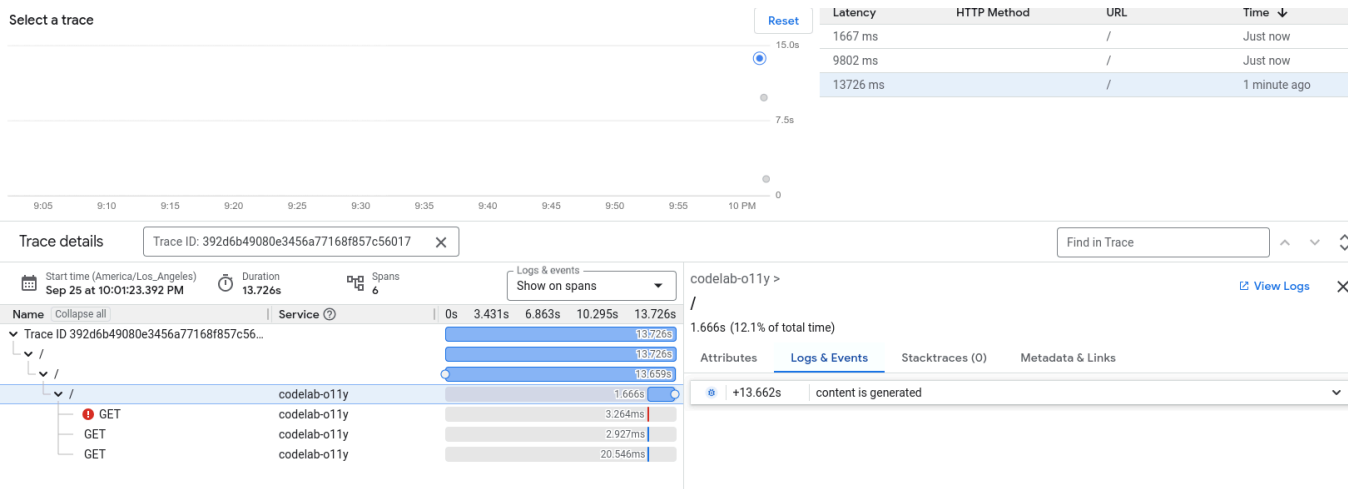
```
Sorry, this is just a placeholder...
```

這表示服務未完成部署作業。請稍候片刻，然後重新整理頁面。最後你會看到以「Fun Dog Facts」(狗狗趣味小知識)開頭的文字，其中包含 10 個狗狗趣味小知識。

如要產生遙測資料，請開啟服務網址。重新整理頁面，同時變更 `?animal=` 參數的值，即可取得不同的結果。

探索應用程式追蹤記錄

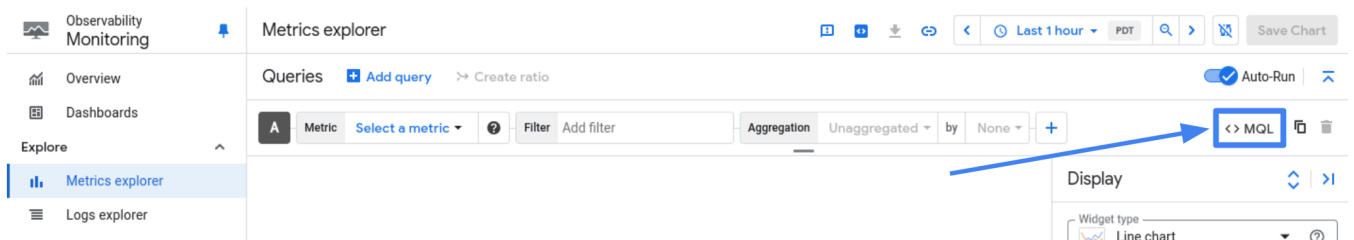
1. 按一下下方按鈕，在 Cloud 控制台中開啟 Trace 探索工具頁面：
2. 選取其中一個最近的追蹤記錄。您應該會看到 5 或 6 個類似下方螢幕截圖的範圍。



3. 找出追蹤事件處理常式 (`fun_facts` 方法) 呼叫的範圍。這是最後一個名為「/」的範圍。
4. 在「追蹤詳細資料」窗格中，選取「記錄和事件」。您會看到與這個特定時距相關聯的應用程式記錄檔。系統會使用追蹤記錄和記錄檔中的追蹤記錄和時距 ID，偵測關聯性。您應該會看到寫入提示的應用程式記錄，以及 Vertex API 的回覆。

探索計數器指標

1. 按一下下方按鈕，在 Cloud 控制台中開啟 Metrics Explorer 頁面：
2. 在查詢建構工具窗格的工具列中，選取名為「<> MQL」或「<> PromQL」的按鈕。如要瞭解按鈕位置，請參閱下圖。



3. 確認「語言」切換按鈕已選取「PromQL」。語言切換鍵位於可格式化查詢的工具列中。
4. 在「查詢」編輯器中輸入查詢：

```
sum(rate(workload_googleapis_com:model_call_counter{monitored_resource="generic_task"}
[ $__interval ] ))
```

5. 按一下「執行查詢」。啟用「自動執行」切換鈕後，系統就不會顯示「執行查詢」按鈕。

11. (選用) 記錄中經過模糊處理的機密資訊

這個步驟是為了教學用途。這個步驟中的指令並非用於執行。

在步驟 10 中，我們記錄了應用程式與 Gemini 模型互動的相關資訊。這項資訊包括動物名稱、實際提示和模型的回應。雖然將這項資訊儲存在記錄檔中應該很安全，但許多其他情況並非如此。提示可能包含使用者不想儲存的個人或其他私密資訊。為解決這個問題，您可以模糊處理寫入 Cloud Logging 的機密資料。為盡量減少程式碼修改，建議採用下列解決方案。

1. 建立 PubSub 主題，用來儲存傳入的記錄項目
 2. 建立[記錄接收器](#)，將擷取的記錄重新導向至 Pub/Sub 主題。
 3. 按照下列步驟建立 [Dataflow](#) 管道，修改重新導向至 PubSub 主題的記錄：
 1. 從 Pub/Sub 主題讀取記錄檔項目
 2. 使用 [DLP 檢查 API](#) 檢查項目酬載是否含有機密資訊
 3. 使用其中一種 [DLP 遮蓋方法](#)，遮蓋酬載中的機密資訊
 4. 將經過模糊處理的記錄項目寫入 Cloud Logging
 4. 部署管道
-

12. (選用) 清除

為避免因使用程式碼研究室的資源和 API 而產生費用，建議您在完成實驗室後進行清理。如要避免付費，最簡單的方法就是刪除您為了本程式碼研究室所建立的專案。

警告：刪除專案會出現以下結果：

- **刪除專案中的所有內容。**如果您使用現有專案來進行本文中的工作，刪除專案將一併刪除其中已完成的所有工作。
- **自訂專案 ID 會消失。**您建立這個專案時，可能已建立日後使用的自訂專案 ID。如要保留使用該專案 ID 的網址 (如 appspot.com 網址)，請從專案刪除選定的資源，不要刪除整個專案。

如果打算探索您建立的應用程式，重複使用專案可節省時間，並避免超出專案配額限制。

1. 如要刪除專案，請在終端機中執行刪除專案指令：

```
PROJECT_ID=$(gcloud config get-value project)
gcloud projects delete ${PROJECT_ID} --quiet
```

刪除 Cloud 專案後，系統就會停止對該專案使用的所有資源和 API 收取費用。您應該會看到以下訊息，其中 `PROJECT_ID` 是您的專案 ID：

```
Deleted [https://cloudresourcemanager.googleapis.com/v1/projects/PROJECT_ID].

You can undo this operation for a limited period by running the command below.
$ gcloud projects undelete PROJECT_ID

See https://cloud.google.com/resource-manager/docs/creating-managing-projects for
information on shutting down projects.
```

2. (選用) 如果收到錯誤訊息，請參閱步驟 5，找出您在實驗室中使用的專案 ID。並代入第一個指令。舉例來說，如果專案 ID 為 `lab-example-project`，指令會是：

```
gcloud projects delete lab-project-id-example --quiet
```

13. 恭喜

在本實驗室中，您已建立生成式 AI 應用程式，並使用 Gemini 模型進行預測。並透過基礎的監控和記錄功能檢測應用程式。您已將應用程式和原始碼的變更內容部署至 [Cloud Run](#)。接著，您可以使用 [Google Cloud Observability](#) 產品追蹤應用程式效能，確保應用程式的可靠性。

如要參與使用者體驗研究，協助我們改善您今天使用的產品，請[按這裡註冊](#)。

以下提供幾種繼續學習的方式：

- 程式碼實驗室：[如何在 Cloud Run 上部署 Gemini 支援的聊天應用程式](#)
 - 程式碼研究室：[如何搭配使用 Gemini 函式呼叫與 Cloud Run](#)
 - [如何使用 Cloud Run Jobs Video Intelligence API 逐場景處理影片](#)
 - 隨選研討會：[Google Kubernetes Engine 新手上路](#)
 - 進一步瞭解如何使用應用程式記錄檔設定[計數器](#)和[分佈](#)指標
 - 使用 [OpenTelemetry Sidecar](#) 寫入 OTLP 指標
 - [參考資料](#)：瞭解如何在 Google Cloud 中使用 Open Telemetry
 - [使用 Go 生成 Open Telemetry 追蹤記錄和指標](#)
-